

Question 1

A company currently stores large volumes of structured and unstructured data in cloud object storage. However, data engineers frequently encounter:

- unexpected schema changes to tables
- failed or conflicting updates during concurrent writes
- downstream reporting inconsistencies caused by partial data updates

The company wants to continue using cloud object storage while improving reliability for analytical workloads.

Which technology best addresses these requirements?

A. Delta Live Tables

B. Delta Lake

C. Parquet

D. Unity Catalog

Correct answer: B

B. Delta Lake

Explanation

The company wants to continue using cloud object storage while improving reliability for production analytical workloads.

Delta Lake extends cloud object storage with capabilities such as:

- ACID transactions
- schema enforcement
- reliable concurrent write handling
- consistent transactional updates

These features help prevent problems such as unexpected schema changes, conflicting writes, and inconsistent downstream reporting caused by partial updates.

Parquet provides only the underlying storage format, while Delta Lake adds the transactional and reliability features needed for production data engineering workloads.

Question 2

A data engineering team attempts to create the following Unity Catalog external table:

```
CREATE TABLE finance.gold.customers
USING DELTA
LOCATION 's3://company-curated/customers';
```

The command fails with a permissions-related error.

The engineering team confirms that:

- the users have `USE CATALOG` on the catalog
- the users have `USE SCHEMA` on the schema
- the target cloud storage path already exists

Which privilege is most likely missing?

- A. READ FILES**
- B. CREATE TABLE**
- C. CREATE EXTERNAL TABLE**
- D. READ VOLUME**

Correct answer: C

C. CREATE EXTERNAL TABLE

Explanation

External tables in Unity Catalog reference data stored outside Unity Catalog managed storage.

When creating external tables:

- the external storage path must already exist
- the storage location must be governed through Unity Catalog external location configurations
- users require privileges related to external table creation

In this scenario:

- the catalog and schema access are already configured
- the storage path exists successfully
- the failure occurs because the users do not have the required privilege to create the external table

This is a common governance and storage integration scenario when working with external cloud storage in Unity Catalog.

Question 3

A data engineering team is troubleshooting a Lakeflow Job that uses a job cluster.

The job fails before any notebook code starts running.

Which issue is the most likely cause of the job failure?

- A. An error in the init script is preventing the cluster from starting successfully.
- B. The Lakeflow Job exceeded the configured task retry limit.
- C. A Bronze table referenced by the notebook does not exist.
- D. A Spark transformation inside the notebook caused excessive shuffle operations.

Correct answer: A

A. An error in the init script is preventing the cluster from starting successfully.

Explanation

Cluster startup failures occur before notebook code or Spark transformations begin executing.

When troubleshooting jobs that fail during startup, engineering teams should investigate cluster-level issues such as:

- init scripts
- cluster configuration
- runtime compatibility
- installed libraries
- permissions

This is different from Spark execution failures, which occur after notebook execution begins.

Understanding whether a failure occurs:

- before cluster startup
- during notebook execution
- or during Spark processing

is an important troubleshooting skill when diagnosing Lakeflow Job failures.

Question 4

A data engineering team manages a large external Delta table in Unity Catalog named `finance.gold.transactions`.

The engineering team wants to migrate the table to a Unity Catalog managed table in order to take advantage of features such as:

- Predictive Optimization
- automatic maintenance operations
- managed storage lifecycle

At the same time, the engineering team wants to avoid recreating the table using `CREATE TABLE AS SELECT` because they want to preserve existing metadata and minimize migration disruption.

Which command best meets these requirements?

A. `CONVERT TO DELTA finance.gold.transactions;`

B.

```
ALTER TABLE finance.gold.transactions  
SET MANAGED;
```

C.

```
ALTER TABLE finance.gold.transactions  
AS MANAGED;
```

D.

```
ALTER TABLE finance.gold.transactions  
CHANGE MANAGED;
```

Correct answer: B

Explanation

Databricks supports converting external Delta tables into Unity Catalog managed tables using:

```
ALTER TABLE table_name SET MANAGED;
```

This approach is useful when engineering teams want to:

- adopt Unity Catalog managed table capabilities
- enable Predictive Optimization
- simplify lifecycle management
- reduce operational maintenance overhead

Compared to recreating tables manually using `CREATE TABLE AS SELECT`, this approach helps:

- preserve existing metadata
- reduce migration complexity
- minimize operational disruption

This feature applies specifically to external Delta tables managed through Unity Catalog.

Question 5

A healthcare organization has created the following row filter function:

```
CREATE FUNCTION hospital_row_filter(hospital_id STRING)
RETURN CASE
  WHEN is_account_group_member('hospital_admins') THEN TRUE
  WHEN is_account_group_member('hospital_a_staff') THEN hospital_id = 'A'
  WHEN is_account_group_member('hospital_b_staff') THEN hospital_id = 'B'
  ELSE FALSE
END;
```

The function should be applied to the table `healthcare.silver.patient_records` so that row-level security is enforced whenever users query the table.

Which SQL statement should be used?

A.

```
ALTER TABLE healthcare.silver.patient_records
SET ROW FILTER hospital_row_filter
ON (hospital_id);
```

B.

```
ALTER TABLE healthcare.silver.patient_records
ADD ROW FILTER hospital_row_filter
ON (hospital_id);
```

C.

```
ALTER TABLE healthcare.silver.patient_records
APPLY ROW FILTER hospital_row_filter
ON (hospital_id);
```

D.

```
ALTER TABLE healthcare.silver.patient_records
ENABLE ROW FILTER hospital_row_filter
ON (hospital_id);
```

Correct answer: A

Explanation

Row filters in Unity Catalog are implemented in two steps:

- Create a function that returns a Boolean value.
- Attach the function to a table using an ALTER TABLE statement.

For example:

```
ALTER TABLE healthcare.silver.patient_records
SET ROW FILTER hospital_row_filter
ON (hospital_id);
```

After the filter is attached, Databricks automatically evaluates the function for each row returned by a query and only returns rows for which the function evaluates to TRUE.

This allows organizations to implement centralized row-level security while storing all records in a single table.

Question 6

A data governance team manages a Unity Catalog schema named `finance.reporting`.

New reporting tables are created in this schema every week.

The team wants analysts to automatically receive query access to any new tables created in the schema, without manually granting `SELECT` on each table.

Which approach best meets this requirement?

- A. Grant `CREATE TABLE` on the `finance.reporting` schema to the analysts group.
- B. Create a user-defined function to automatically grant privileges whenever new tables are created.
- C. Grant `USE SCHEMA` on the `finance.reporting` schema to the analysts group.
- D. Grant `SELECT` on the `finance.reporting` schema to the analysts group.

Correct answer: D

D. Grant `SELECT` on the `finance.reporting` schema to the analysts group.

Explanation

Unity Catalog supports privilege inheritance through its securable object hierarchy.

In this scenario:

- new tables are created regularly
- analysts should automatically receive query access
- the governance team wants to avoid manually granting privileges on each table

Granting `SELECT` on the schema allows child tables within the schema to inherit the privilege automatically.

This helps simplify governance administration and reduces operational overhead in environments with frequently created datasets.

Question 7

A data engineering team deploys the same Declarative Automation Bundle across development, test, and production environments.

The engineering team wants:

- smaller clusters in development
- larger clusters in production
- different job schedules in each environment
- the same underlying codebase to be reused across all environments

Which approach best meets these requirements?

- A.** Maintain separate Declarative Automation Bundle configurations for development, test, and production environments.
- B.** Maintain separate deployment branches for development, test, and production environments with environment-specific configurations.
- C.** Parameterize cluster and schedule settings directly inside notebooks and pass environment values during execution.
- D.** Use target-specific overrides within the Declarative Automation Bundle to customize environment-specific deployment settings.

Correct answer: D

D. Use target-specific overrides within the Declarative Automation Bundle to customize environment-specific deployment settings.

Explanation

Declarative Automation Bundles support environment-specific deployment behavior through:

- deployment targets
- variables
- target-specific overrides

This allows teams to:

- reuse the same codebase across environments
- customize deployment settings per environment
- avoid maintaining duplicate deployment configurations

Common environment-specific overrides include:

- cluster configurations
- schedules
- catalog names
- workspace settings

Using target-specific overrides improves:

- deployment consistency
- CI/CD maintainability
- environment isolation
- operational scalability

This approach is commonly used when promoting the same Lakeflow Jobs and Lakeflow Spark Declarative Pipelines deployments across development, test, and production environments.

Question 8

A company uses a Lakeflow Connect database connector to continuously ingest changes from a PostgreSQL operational database into Delta tables.

The engineering team wants to understand which Lakeflow Connect component is responsible for continuously capturing database snapshots, change logs, and metadata from the source database and writing the extracted data into staging storage.

Which component performs this responsibility?

A. Lakeflow Spark Declarative Pipelines

B. Staging storage

C. Lakeflow Jobs

D. Ingestion gateway

Correct answer: D

D. Ingestion gateway

Explanation

Lakeflow Connect database connectors use multiple components to support managed CDC-style ingestion workflows.

The ingestion gateway is responsible for:

- continuously reading database snapshots and change logs
- capturing source metadata
- writing extracted data into staging storage before downstream processing

This architecture helps support scalable and reliable incremental ingestion from operational databases into Databricks.

Question 9

A company uses Auto Loader to continuously ingest files from cloud object storage into Bronze Delta tables.

Over time, the source directory has grown to millions of files, and the engineering team notices increasing overhead when scanning cloud storage locations to detect newly arrived files.

The team wants to:

- reduce file discovery overhead
- improve ingestion scalability
- continue processing newly arriving files incrementally

Which Auto Loader configuration is generally most appropriate?

- A.** Use checkpoint recovery mode
- B.** Use schema inference mode
- C.** Use directory listing mode
- D.** Use file notification mode

Correct answer: D

- D.** Use file notification mode

Explanation

The scenario focuses on scaling Auto Loader ingestion for very large cloud storage directories containing millions of files.

As storage locations grow, repeatedly scanning directories for new files can increase ingestion overhead.

File notification mode improves scalability by:

- reducing directory scanning operations
- using cloud-native event notifications for new file discovery
- improving ingestion efficiency for large-scale workloads
- supporting scalable continuous ingestion pipelines

This makes file notification mode a strong fit for high-volume ingestion scenarios operating at scale.

Question 10

A data engineering team is building a batch ingestion pipeline that incrementally loads CSV files from cloud object storage into a Bronze Delta table.

The team wants the ingestion process to:

- load only newly arrived files
- avoid reprocessing previously loaded files
- use a SQL-based ingestion approach

Which code snippet best meets these requirements?

A.

```
COPY INTO retail.bronze.orders
FROM '/Volumes/retail/landing/orders'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true')
```

B.

```
MERGE INTO retail.bronze.orders t
USING csv.'/Volumes/retail/landing/orders' s
ON t.order_id = s.order_id
```

C.

```
INSERT OVERWRITE retail.bronze.orders
SELECT *
FROM csv.'/Volumes/retail/landing/orders'
```

D.

```
CREATE OR REPLACE TABLE retail.bronze.orders AS
SELECT *
FROM csv.'/Volumes/retail/landing/orders'
```

Correct answer: A

Explanation

COPY INTO is designed for incremental ingestion of files from cloud object storage into Delta tables.

It supports:

- SQL-based ingestion
- tracking previously loaded files
- avoiding duplicate file ingestion
- recurring batch ingestion workflows

This makes it well suited for Bronze layer ingestion pipelines where new files arrive continuously over time.

Question 11

A financial services company regularly transfers CSV files via SFTP to provide regulators with quarterly reports. This approach is slow, error-prone, and regulators often work with outdated data.

The company has enabled Unity Catalog in Databricks and wants to provide regulators with **live, read-only access** to selected Lakehouse tables without creating copies of the data.

Which feature should they use?

A. Unity Catalog Volumes

B. External Locations

C. Delta Sharing

D. Lakehouse Federation

Correct answer: C

C. Delta Sharing

Explanation

The company wants to replace periodic file transfers with a solution that provides regulators with access to current data while avoiding data duplication.

Delta Sharing is designed for this scenario because it:

- provides secure, read-only access to shared datasets
- allows recipients to access the latest available data
- avoids creating additional copies of the data
- supports sharing across organizations and platforms

This makes Delta Sharing a common choice when organizations need to share governed Lakehouse data with external partners, regulators, suppliers, or customers.

Question 12

A data engineering team is building a Lakeflow Job to process financial reconciliation data.

The workflow must run:

- on the 1st and 16th day of every month
- at both 10:00 AM and 4:00 PM UTC

Which approach best meets these requirements?

- A.** Configure a scheduled trigger using a CRON expression.
- B.** Configure a scheduled trigger using the standard scheduling options in the Lakeflow Jobs UI.
- C.** Configure a file arrival trigger on the reconciliation data location.
- D.** Configure a table update trigger on the reconciliation tables.

Correct answer: A

- A.** Configure a scheduled trigger using a CRON expression.

Explanation

Lakeflow Jobs supports scheduled triggers using CRON expressions for advanced recurring workflow schedules.

CRON scheduling is useful when workflows must run:

- on specific days
- at multiple times during the day
- using complex recurring calendar patterns

In this scenario, the workflow must run:

- twice per day
- on specific days of the month

This type of advanced recurring schedule is best implemented using a CRON expression rather than standard UI scheduling options or data-driven triggers.

Question 13

A data engineering team manages a Unity Catalog table named `sales.gold.orders`.

Users from different regional teams should only be able to view rows belonging to their assigned sales region when querying the table directly.

For example:

- UK users should only see UK orders
- US users should only see US orders

The engineering team wants this filtering logic to be enforced automatically whenever users query the table.

Which Unity Catalog feature best meets this requirement?

- A. Row filters**
- B. Dynamic views
- C. Adaptive Query Execution
- D. Column masks

Correct answer: A

A. Row filters

Explanation

Row filters in Unity Catalog provide centralized row-level security enforcement.

They are commonly used when:

- users should only see records for their own region
- departments should access only their own business data
- sensitive records must be filtered based on user identity or group membership

In this scenario:

- all users query the same table directly

- the visible rows should change automatically depending on the user
- filtering must be centrally enforced

Row filters are designed specifically for this type of governance requirement.

Unlike dynamic views, row filters allow security policies to be enforced directly on the table itself without requiring users to query separate views.

Question 14

A data engineering team is building a Medallion architecture using Lakeflow Spark Declarative Pipelines.

The engineering team wants:

- Bronze datasets to continuously ingest newly arriving raw event data
- Silver datasets to incrementally process and refine the incoming event stream
- Gold datasets to provide precomputed aggregations for BI dashboards and reporting workloads

Which architecture best matches these requirements?

A. Bronze: View

Silver: Materialized view

Gold: Streaming table

B. Bronze: Streaming table

Silver: Streaming table

Gold: Materialized view

C. Bronze: Streaming table

Silver: View

Gold: Streaming table

D. Bronze: Materialized view

Silver: Streaming table

Gold: View

Correct answer: B

B. Bronze: Streaming tableSilver: Streaming tableGold: Materialized view

Explanation

In Lakeflow Spark Declarative Pipelines, different dataset types are optimized for different stages of the Medallion architecture.

Streaming tables are commonly used in Bronze and Silver layers because they support incremental processing of continuously arriving data.

This makes them well suited for:

- streaming ingestion

- incremental cleansing
- near real time transformations

Materialized views are commonly used in Gold layers because they store precomputed query results that improve dashboard and analytical query performance.

Question 15

A retail company is building a Gold customer analytics dataset using PySpark.

The analytics team needs an estimate of the number of unique customers who placed orders during the last 12 months. The dataset contains billions of records, and the engineering team wants a faster aggregation with lower resource usage instead of performing a full exact distinct count.

Which aggregation function best meets this requirement?

A.

```
sales_df.select(  
    count_distinct("customer_id")  
)
```

B.

```
sales_df.select(  
    count("customer_id")  
)
```

C.

```
sales_df.select(  
    count(distinct("customer_id"))  
)
```

D.

```
sales_df.select(  
    approx_count_distinct("customer_id")  
)
```

Correct answer: D

Explanation

`approx_count_distinct()` is used when an approximate estimate of unique values is acceptable and performance is more important than exact precision.

Using:

```
approx_count_distinct("customer_id")
```

allows Spark to estimate the number of unique customer IDs without performing a full exact distinct calculation across the entire dataset.

Compared to an exact distinct count, this approach:

- reduces shuffle overhead
- lowers memory usage
- improves aggregation performance on very large datasets

It is commonly used in large-scale analytical workloads where approximate cardinality estimates are sufficient for reporting and trend analysis.

Question 16

A data engineering team wants to reduce operational overhead for interactive analytics workloads in Databricks.

The team prefers a compute option that:

- automatically manages infrastructure provisioning
- scales resources based on workload demand
- reduces manual compute management

Which compute option best meets these requirements?

- A. Serverless compute**
- B. All-purpose compute
- C. Compute Pools
- D. Elastic Compute

Correct answer: A

A. Serverless compute

Explanation

The scenario focuses on reducing operational overhead for interactive analytics workloads.

Serverless compute helps achieve this by:

- automatically provisioning infrastructure
- scaling resources based on workload demand
- reducing manual compute management responsibilities

This makes serverless compute well suited for teams that want a simplified operational experience while still supporting scalable analytics workloads.

The question also tests familiarity with actual Databricks compute terminology versus plausible but non-existent platform terminology.

Question 17

A media company receives continuously arriving nested JSON clickstream files in a Unity Catalog Volume.

The data engineering team wants the Auto Loader pipeline to:

- incrementally process newly arriving files
- track schema information over time
- stop the ingestion pipeline if unexpected new columns appear in incoming data

Which code snippet best meets these requirements?

A.

```
df = (  
    spark.readStream  
        .format("json")  
        .option("cloudFiles.schemaLocation",  
            "/Volumes/media/checkpoints/clickstream_schema")  
        .option("cloudFiles.schemaEvolutionMode", "failOnNewColumns")  
        .load("/Volumes/media/landing/clickstream")  
)
```

B.

```
df = (  
    spark.readStream  
        .format("cloudFiles")  
        .option("cloudFiles.format", "json")  
        .option("cloudFiles.schemaLocation",  
            "/Volumes/media/checkpoints/clickstream_schema")  
        .option("cloudFiles.schemaEvolutionMode", "failOnNewColumns")  
        .load("/Volumes/media/landing/clickstream")  
)
```

C.

```
df = (  
    spark.read  
        .format("cloudFiles")  
        .option("cloudFiles.format", "json")  
        .option("cloudFiles.schemaEvolutionMode", "failOnNewColumns")  
        .load("/Volumes/media/landing/clickstream")  
)
```

D.

```
df = (  
    spark.readStream  
        .format("cloudFiles")  
        .option("format", "json")  
        .option("schemaLocation", "/Volumes/media/checkpoints/clickstream_schema")  
        .option("schemaEvolutionMode", "failOnNewColumns")  
        .load("/Volumes/media/landing/clickstream")  
)
```

Correct answer: B

Explanation

Auto Loader uses the `cloudFiles` source with `spark.readStream` to incrementally process files as they arrive in cloud object storage.

For schema tracking and schema control:

- `cloudFiles.schemaLocation` stores schema information across ingestion runs
- `cloudFiles.schemaEvolutionMode` controls how Auto Loader handles newly detected columns
- `failOnNewColumns` causes the stream to fail when unexpected new columns are detected

This is useful when teams want strict control over schema changes in production ingestion pipelines.

Question 18

A data engineering team wants to deploy a Declarative Automation Bundle to the production environment as part of an automated CI/CD workflow.

Which Databricks CLI command best meets this requirement?

- A. `databricks bundle deploy --env prod`
- B. `databricks bundle deploy`
- C. `databricks bundle deploy --target prod`
- D. `databricks bundle deploy prod`

Correct answer: C

C. `databricks bundle deploy --target prod`

Explanation

The Databricks CLI supports Declarative Automation Bundle deployment workflows through commands such as:

```
databricks bundle deploy --target <target_name>
```

Deployment targets define environment-specific settings such as:

- workspace configuration
- cluster settings
- schedules
- catalog names
- environment variables

Using deployment targets allows teams to:

- reuse the same codebase across environments
- standardize deployments
- support CI/CD workflows
- simplify environment promotion

This approach is commonly used when deploying Lakeflow Jobs and Lakeflow Spark Declarative Pipelines across development, test, and production environments.

Question 19

A data engineering team manages a large IoT analytics workload in Databricks.

The workload experiences changing query patterns over time, and the engineering team finds that maintaining an effective partitioning strategy has become increasingly difficult.

The engineering team wants a more flexible data layout approach that can adapt more efficiently to evolving query patterns without relying heavily on static partitioning strategies.

Which Databricks feature best meets these requirements?

- A. Liquid Clustering
- B. Adaptive Query Execution
- C. Broadcast joins
- D. Predictive Optimization

Correct answer: A

A. Liquid Clustering

Explanation

Liquid Clustering is a Delta Lake optimization feature designed to improve data layout flexibility.

Traditional partitioning strategies can become difficult to maintain when:

- query patterns change frequently
- data access patterns evolve over time
- partition design requirements become unpredictable

Liquid Clustering helps address these challenges by:

- providing more flexible clustering behavior
- reducing dependence on static partitioning strategies
- improving query performance for evolving workloads

Liquid Clustering is particularly useful for:

- large analytics workloads
- evolving query patterns
- high-volume ingestion environments

Question 20

A logistics company receives continuous vehicle telemetry events through an Apache Kafka topic named `vehicle_telemetry`.

The data engineering team wants to build a streaming ingestion pipeline that:

- reads events continuously from Kafka
- subscribes to the correct Kafka topic
- starts processing only new events arriving after the stream starts

Which code snippet best meets these requirements?

A.

```
df = (  
    spark.readStream  
        .format("kafka")  
        .option("kafka.bootstrap.servers", "broker1:9092")  
        .option("subscribe", "vehicle_telemetry")  
        .option("startingOffsets", "latest")  
        .load()  
)
```

B.

```
df = (  
    spark.readStream  
        .format("json")  
        .option("kafka.bootstrap.servers", "broker1:9092")  
        .option("subscribe", "vehicle_telemetry")  
        .option("startingOffsets", "latest")  
        .load()  
)
```

C.

```
df = (  
    spark.readStream  
        .format("kafka")  
        .option("kafka.bootstrap.servers", "broker1:9092")  
        .option("subscribe", "vehicle_telemetry")  
        .option("startingOffsets", "earliest")  
        .load()  
)
```

D.

```
df = (  
    spark.readStream  
        .format("kafka")  
        .option("bootstrap.servers", "broker1:9092")  
        .option("topic", "vehicle_telemetry")  
        .option("startingOffset", "latest")  
        .load()  
)
```

Correct answer: A

Explanation

For streaming ingestion from Kafka in Databricks, the pipeline should use Spark Structured Streaming with the Kafka source.

The correct configuration includes:

- `spark.readStream` for streaming ingestion
- `.format("kafka")` to read from Kafka
- `kafka.bootstrap.servers` to identify the Kafka broker
- `subscribe` to read from a Kafka topic
- `startingOffsets = "latest"` to process only events arriving after the stream starts

This configuration is appropriate when the pipeline should continuously ingest new Kafka events into downstream processing.

Question 21

A data engineering team manages a Unity Catalog table named `sales.gold.orders`.

The engineering team wants to implement the following requirements:

- regional managers should only see orders for their own region
- non-manager users should not see the `profit_margin` column
- users should query a single governed object instead of accessing the base table directly
- the filtering and masking logic should be implemented together in one layer

Which approach best meets these requirements?

- A. Column masks
- B. Security views
- C. Row filters
- D. Dynamic views

Correct answer: D

D. Dynamic views

Explanation

Dynamic views are commonly used when engineering teams want to:

- combine row-level and column-level security logic
- expose a governed interface instead of direct table access
- customize query results based on user or group identity

In this scenario:

- users should not query the base table directly
- row filtering and column masking should be implemented together

- the engineering team wants a single governed access layer

Dynamic views are well suited for this type of governance design because they allow security logic to be embedded directly within the view definition.

Unlike row filters or column masks, which apply directly to tables, dynamic views provide a flexible governed abstraction layer on top of underlying data.

Question 22

A data engineering team is building a Lakeflow Job to process regional sales data.

The same notebook logic must execute separately for multiple regions:

- UK
- US
- Germany
- France
- Japan

The engineering team wants to:

- reuse the same notebook task logic
- avoid creating separate notebook tasks for each region
- dynamically execute the workflow once per region

Which approach best meets these requirements?

- A.** Create separate notebook tasks manually for each region inside the Lakeflow Job.
- B.** Configure a Run Job task for each region and pass the region value as a job parameter.
- C.** Configure an If/else task to route execution based on the selected region.
- D.** Configure a For each task to iterate through the list of regions and execute the notebook task dynamically with the region value.

Correct answer: D

D. Configure a For each task to iterate through the list of regions and execute the notebook task dynamically with the region value.

Explanation

Lakeflow Jobs supports iterative workflow execution using For each tasks.

A For each task allows the same downstream task to execute repeatedly using different parameter values from a collection.

In this scenario:

- the same notebook logic must run for multiple regions
- creating separate tasks for each region would duplicate orchestration logic
- the workflow should scale easily if additional regions are added later

Using a For each task improves:

- workflow maintainability
- orchestration scalability
- parameterized execution
- DAG simplicity

This pattern is commonly used when identical processing logic must run across multiple regions, datasets, or customers.

Question 23

A data engineering team is building a new analytics solution in Databricks.

Several developers work collaboratively in notebooks and frequently run code interactively while developing and testing transformations.

The development environment requires:

- custom init scripts to run during compute startup
- a proprietary third-party library to be installed on all nodes
- consistent environment configuration before notebook execution begins

Which compute option is most appropriate for this workload?

- A.** All-purpose Compute
- B.** Serverless Compute
- C.** SQL Warehouse
- D.** Job Compute

Correct answer: A

A. All-purpose Compute

Explanation

The key requirements in this scenario are:

- collaborative interactive notebook development
- custom init scripts during startup
- proprietary libraries installed on all nodes
- consistent environment configuration

While Serverless Compute simplifies infrastructure management, some workloads require startup customization before notebook execution begins.

Job Compute can support this customization, but it is optimized for scheduled and automated execution rather than collaborative development.

Because multiple developers are working interactively in notebooks and the environment requires custom configuration, All-purpose Compute is the most appropriate choice. It provides a shared development environment while allowing engineering teams to control runtime configuration and installed libraries.

Question 24

A data engineering team runs large Spark-based batch ETL pipelines overnight in Databricks to process clickstream and transaction data. The pipelines execute for several hours and process terabytes of data each night.

The workloads are designed to be fault tolerant, and occasional task retries do not impact overall pipeline completion.

Management wants to reduce infrastructure cost for these workloads while still maintaining scalable distributed processing capacity.

Which compute configuration best supports these requirements?

- A. Enable Photon for all workloads
- B. Use a smaller fixed-size cluster
- C. Use spot instances for worker nodes
- D. Use on-demand instances for worker nodes

Correct answer: C

C. Use spot instances for worker nodes

Explanation

The scenario focuses on reducing infrastructure cost for large-scale distributed batch workloads that can tolerate occasional interruptions.

Spot instances provide discounted compute capacity compared to standard on-demand instances. Because Spark workloads are distributed and fault tolerant, failed tasks can often be retried automatically when spot instances are reclaimed.

This makes spot instances a cost-effective choice for:

- large overnight ETL pipelines
- retry-tolerant distributed workloads
- non-interactive batch processing jobs

Using spot instances for worker nodes helps organizations reduce compute cost while still maintaining scalable distributed processing capacity.

Question 25

A finance organization needs to ingest data nightly from an enterprise Oracle database into Delta tables in Databricks.

The engineering team wants to:

- execute custom SQL queries against the source database
- apply custom transformation logic inside notebooks

Which approach best fits these requirements?

- A. Use Auto Loader for database ingestion
- B. Use JDBC ingestion inside notebooks
- C. Use COPY INTO from the Oracle database
- D. Use Delta Sharing recipients

Correct answer: B

- B. Use JDBC ingestion inside notebooks

Explanation

The scenario focuses on a custom notebook-based ingestion workflow for a relational database source.

JDBC ingestion inside notebooks is commonly used when teams need:

- custom SQL extraction logic
- notebook-based transformation processing
- direct connectivity to relational databases

This provides engineers with flexibility to control extraction and transformation behavior directly within Databricks notebooks.

Question 26

A data engineering team enables Predictive Optimization for several Unity Catalog tables used in a reporting workload.

After a few days, the engineering team notices that some tables are automatically optimized while others are not.

Why is this behavior occurring?

- A. Predictive Optimization only supports Unity Catalog managed tables.
- B. Predictive Optimization requires all tables to be partitioned before optimization can occur.
- C. Predictive Optimization only runs on tables referenced by Lakeflow Jobs.
- D. Predictive Optimization only supports Delta tables that use Liquid Clustering.

Correct answer: A

A. Predictive Optimization only supports Unity Catalog managed tables.

Explanation

Predictive Optimization automatically performs maintenance operations such as:

- OPTIMIZE
- VACUUM
- ANALYZE

for Unity Catalog managed tables.

This helps:

- improve query performance
- reduce manual operational effort
- maintain efficient Delta table layouts

Predictive Optimization does not automatically manage external tables because Databricks does not fully manage the underlying storage lifecycle for those tables.

Understanding the difference between:

- managed tables
- external tables

is important when designing optimization and maintenance strategies in Databricks.

Question 27

A data engineering team manages deployments using custom Databricks REST API scripts. The process is slow, difficult to maintain, and requires engineers to work directly with low-level API calls.

The engineering team wants:

- a native and supported deployment approach
- repeatable deployments across development, test, and production environments
- a simpler way to package and deploy Databricks projects within CI/CD workflows

Which approach best meets these requirements?

- A.** Continue using Databricks REST APIs and expand the deployment scripts for each environment.
- B.** Use DBX from Databricks Labs to package and deploy Databricks projects.
- C.** Use Declarative Automation Bundles to package and deploy Databricks projects across environments.
- D.** Use Terraform to deploy Lakeflow Jobs, Lakeflow Spark Declarative Pipelines, and notebooks as infrastructure resources.

Correct answer: C

- C.** Use Declarative Automation Bundles to package and deploy Databricks projects across environments.

Explanation

Declarative Automation Bundles provide a native deployment framework for Databricks projects.

Bundles allow teams to:

- package workspace assets together
- standardize deployments
- reuse the same codebase across environments
- support CI/CD workflows
- reduce deployment complexity

Bundles commonly manage:

- Lakeflow Jobs
- Lakeflow Spark Declarative Pipelines
- notebooks
- deployment configuration
- environment-specific settings

Compared to custom REST API scripting, Declarative Automation Bundles provide a more maintainable, repeatable, and CI/CD-friendly deployment approach for modern Databricks projects.

Question 28

A data engineering team is designing workloads for different usage patterns in Databricks.

The team has the following requirements:

- engineers need shared notebook environments with development-specific libraries already installed
- production ETL pipelines should run in isolated environments
- lightweight experimentation workloads should minimize infrastructure cost

Which combination of compute options best matches these requirements?

- A.** Job compute for engineers, all-purpose compute for ETL pipelines, SQL warehouses for lightweight experimentation
- B.** Single-node compute for engineers, serverless compute for ETL pipelines, multi node all-purpose compute for lightweight experimentation
- C.** All-purpose compute for engineers, job compute for ETL pipelines, single-node compute for lightweight experimentation
- D.** SQL warehouses for engineers, single-node compute for ETL pipelines, job compute for lightweight experimentation

Correct answer: C

C. All-purpose compute for engineers, job compute for ETL pipelines, single-node compute for lightweight experimentation

Explanation

Different Databricks compute options are optimized for different workload types.

This scenario requires matching workload characteristics to the most appropriate compute choice:

- all-purpose compute for collaborative notebook development with shared libraries
- job compute for isolated scheduled production execution
- single-node compute for lightweight or lower-cost experimentation workloads

Choosing the correct compute type helps improve collaboration, workload isolation, scalability, and cost efficiency across the platform.

Question 29

A manufacturing company is building a Silver machine alert dataset using PySpark.

The team has two DataFrames:

- machines_df: contains registered machine details
- alerts_df: contains machine alert events

The business requires the final Silver dataset to include every alert event, even if some alerts reference machines that no longer exist in the machine registry. If machine details exist, they should be included alongside the alert information.

Which join best meets this requirement?

A. `silver_df = machines_df.crossJoin(alerts_df)`

B.

```
silver_df = machines_df.join(
    alerts_df,
    machines_df.machine_id == alerts_df.machine_id,
    "right"
)
```

C.

```
silver_df = machines_df.join(
    alerts_df,
    machines_df.machine_id == alerts_df.machine_id,
    "left"
)
```

D.

```
silver_df = machines_df.join(
    alerts_df,
    machines_df.machine_id == alerts_df.machine_id,
    "inner"
)
```

Correct answer: B

Explanation

A right join preserves all rows from the right-side DataFrame in the join operation.

In this scenario:

```
machines_df.join(alerts_df, join_condition, "right")
```

ensures that every alert event from alerts_df is included in the final result, even when no matching machine record exists in machines_df.

If matching machine details are found, the related machine columns are populated. Otherwise, the machine-related columns contain NULL values.

Right joins are useful when the business requirement is to preserve all records from the right-side dataset during enrichment operations.

Question 30

An IoT data engineering team is troubleshooting a Lakeflow Job that joins a large `sensor_readings` table with a small `device_metadata` lookup table.

While reviewing the Spark UI, the engineering team notices:

- both tables are being shuffled during the join
- the `device_metadata` table is only a few megabytes in size
- the join stage is taking longer than expected

The engineering team wants to reduce network shuffle during the join operation.

Which optimization is the most appropriate?

- A.** Enable Liquid Clustering on the `sensor_readings` table.
- B.** Run `OPTIMIZE` on both tables before executing the join.
- C.** Increase the number of shuffle partitions.
- D.** Use a broadcast join for the `device_metadata` lookup table.

Correct answer: D

- D.** Use a broadcast join for the `device_metadata` lookup table.

Explanation

Broadcast joins are commonly used when joining:

- a very large table
- with a much smaller lookup or dimension table

Instead of shuffling both tables across executors, Spark broadcasts the smaller table to all executors processing the larger table.

This helps:

- reduce network shuffle
- reduce shuffle read/write activity

- improve join performance
- reduce stage execution time

Question 31

A data engineering team has separated its Medallion architecture into three independent Lakeflow Jobs:

- bronze_ingestion_job
- silver_transformation_job
- gold_reporting_job

The engineering team wants:

- the Silver job to execute automatically after the Bronze job completes successfully
- the Gold job to execute automatically after the Silver job completes successfully

Which approach best meets these requirements?

- A.** Create a master notebook that executes all Bronze, Silver, and Gold notebooks sequentially using %run commands.
- B.** Configure separate scheduled triggers for each Lakeflow Job with considerable delay between them
- C.** Create another Lakeflow Job to include all the tasks from Bronze, Silver, and Gold Jobs
- D.** Create another Lakeflow Job with Run Job tasks to execute the Bronze, Silver, and Gold Lakeflow Jobs sequentially.

Correct answer: D

- D.** Create another Lakeflow Job with Run Job tasks to execute the Bronze, Silver, and Gold Lakeflow Jobs sequentially.

Explanation

Lakeflow Jobs supports Run Job tasks for chaining independent jobs together into larger orchestration workflows.

This pattern is useful when:

- Medallion layers are maintained independently
- reusable orchestration components are required

- workflows should remain modular

In this scenario:

- Bronze, Silver, and Gold orchestration logic remain separated
- downstream jobs execute only after upstream jobs complete successfully
- orchestration logic is centralized without duplicating notebook execution logic

Using Run Job tasks improves:

- workflow modularity
- orchestration maintainability
- job reuse
- dependency-driven execution across independent workflows

Question 32

A data engineering team is troubleshooting a Spark workload running large joins and aggregations on a multi-node cluster.

The team notices:

- very uneven task execution times
- some executors finishing quickly while others run significantly longer
- overall slow completion of shuffle stages

After reviewing the Spark UI, the engineers suspect that a small number of partitions contain significantly more data than others.

Which issue is the workload most likely experiencing?

- A.** Broadcast threshold overflow
- B.** Driver memory fragmentation
- C.** Excessive caching
- D.** Data skew

Correct answer: D

D. Data skew

Explanation

Data skew occurs when data is distributed unevenly across Spark partitions.

During shuffle operations such as:

- joins
- aggregations
- groupBy operations

Spark creates tasks to process individual partitions. If a small number of partitions contain significantly more data than others, those tasks take much longer to complete.

This often results in:

- uneven task durations
- executors sitting idle while waiting for large skewed tasks
- slower overall job completion

Question 33

A team uses Lakeflow Jobs to refresh dashboards for three business units.

Each dashboard refresh is implemented as a separate task, and all three tasks run in parallel.

The team wants to receive a single success notification only after all dashboard refreshes have completed successfully. They do not want to create additional tasks that perform no business processing.

What is the best way to configure the workflow?

- A.** Configure a job-level success notification for the Lakeflow Job.
- B.** Configure a webhook notification inside each dashboard refresh notebook.
- C.** Create a final notification task that depends on all three dashboard refresh tasks and configure the notification on that task.
- D.** Configure a success notification on each dashboard refresh task.

Correct answer: A

- A.** Configure a job-level success notification for the Lakeflow Job.

Explanation

Lakeflow Jobs supports notifications at both the task level and the job level.

When a workflow contains multiple parallel tasks and the requirement is to receive a single notification based on the outcome of the entire workflow, a job-level notification is the most appropriate configuration.

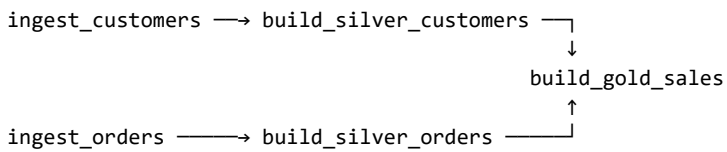
This approach:

- avoids duplicate notifications
- reflects the status of the entire workflow
- avoids introducing unnecessary orchestration tasks
- simplifies operational monitoring

It is commonly used for production workflows that contain multiple parallel branches but require a single completion notification.

Question 34

A data engineering team has a Lakeflow Job with the following task dependencies:



During one execution:

- build_silver_customers completed successfully
- build_silver_orders failed after exhausting all configured retries

Later, the engineering team notices that build_gold_sales still executed successfully but produced only partial downstream data.

Which Run if conditions on the dependencies to build_gold_sales could have caused this behavior?

Select TWO answers.

- A. All done
- B. At least one succeeded
- C. All failed
- D. All succeeded

Correct answers: A, B

- A. All done
- B. At least one succeeded

Explanation

Lakeflow Jobs supports configurable Run if conditions on task dependencies. These conditions determine when downstream tasks execute based on the outcome of upstream tasks.

Common Run if conditions include:

- **All succeeded:** All upstream dependencies must complete successfully. This is the default behavior.
- **At least one succeeded:** The downstream task executes if at least one upstream dependency succeeds.

- **None failed:** No upstream dependencies failed, and at least one dependency executed successfully.
- **All done:** The downstream task executes after all upstream tasks finish, regardless of success or failure outcomes.
- **At least one failed:** The downstream task executes if at least one upstream dependency fails.
- **All failed:** The downstream task executes only if all upstream dependencies fail.

In this scenario:

- build_silver_customers succeeded
- build_silver_orders failed
- build_gold_sales still executed

This behavior could occur with:

- At least one succeeded
- All done

Both configurations can allow downstream tasks to execute even when upstream failures occur, potentially resulting in partial downstream data.

Question 35

A manufacturing company ingests nested IoT telemetry data into a Bronze table using PySpark.

Each record contains a nested array column named `sensor_batches`.

For example:

```
{
  "machine_id": "M101",
  "sensor_batches": [[12.1, 13.5], [14.2, 11.8]]
}
```

The engineering team wants the Silver table to contain one row per individual sensor reading as below.

machine_id	sensor_reading
M101	12.1
M101	13.5
M101	14.2
M101	11.8

Which transformation best meets this requirement?

A.

```
silver_df = bronze_df.withColumn(
  "sensor_reading",
  explode(col("sensor_batches"))
)
```

B.

```
silver_df = bronze_df.withColumn(
  "sensor_reading",
  flatten(explode(col("sensor_batches")))
)
```

C.

```
silver_df = bronze_df.withColumn(
  "sensor_reading",
  flatten(col("sensor_batches"))
)
```

D.

```
silver_df = bronze_df.withColumn(
  "sensor_reading",
  explode(flatten(col("sensor_batches")))
)
```

Correct answer: D

Explanation

The source data contains an array of arrays:

```
[[12.1, 13.5], [14.2, 11.8]]
```

Using:

```
flatten(col("sensor_batches"))
```

first converts the nested arrays into a single array:

```
[12.1, 13.5, 14.2, 11.8]
```

Then:

```
explode(...)
```

creates one output row for each individual sensor reading:

```
12.1
```

```
13.5
```

```
14.2
```

```
11.8
```

This transformation pattern is commonly used when processing nested JSON or semi-structured Bronze data before writing analytics-ready Silver datasets.

Question 36

A data engineering team manages a Declarative Automation Bundle (DAB) that deploys the same Lakeflow Job to development, test, and production environments.

The Lakeflow Job reads data from a Unity Catalog catalog named `dev_catalog` in development, `test_catalog` in test, and `prod_catalog` in production.

The team wants to maintain a single codebase and a single bundle definition while ensuring the correct catalog is used in each environment during deployment.

Which approach best meets these requirements?

- A. Create three Git branches named `dev`, `test`, and `prod`, each containing a different catalog name in the bundle configuration.
- B. Maintain three separate copies of the Declarative Automation Bundle, one for each environment.
- C. Define a bundle variable for the catalog name and use target-specific overrides for each environment.
- D. Store the catalog name directly in the notebook code and manually edit it before each deployment.

Correct answer: C

- C. Define a bundle variable for the catalog name and use target-specific overrides for each environment.

Explanation

A key goal of CI/CD is to promote the same application through multiple environments while minimizing code differences.

Declarative Automation Bundles support this by allowing teams to define variables and target-specific overrides. This enables a single bundle definition to be reused across development, test, and production environments while supplying different configuration values where required.

Common examples include:

- catalog names
- schema names
- workspace URLs
- storage locations
- job parameters

This approach helps reduce duplication, simplify deployments, and maintain consistency across environments.

Question 37

A data engineering team is ingesting JSON files from cloud object storage into a Bronze DataFrame using Spark. Each source file contains a single JSON document formatted across multiple lines rather than one JSON record per line.

The team notices that the ingestion pipeline reads the files incorrectly when using the default JSON reader behavior.

Which code snippet best supports ingesting these files correctly?

A.

```
df = (  
    spark.read  
        .format("json")  
        .option("jsonDoc", "true")  
        .load("/Volumes/retail/landing/orders")  
)
```

B.

```
df = (  
    spark.read  
        .format("json")  
        .option("inferSchema", "true")  
        .load("/Volumes/retail/landing/orders")  
)
```

C.

```
df = (  
    spark.read  
        .format("json")  
        .option("multiline", "true")  
        .load("/Volumes/retail/landing/orders")  
)
```

D.

```
df = (  
    spark.read  
        .format("json")  
        .option("mergeSchema", "true")  
        .load("/Volumes/retail/landing/orders")  
)
```

Correct answer: C

Explanation

By default, Spark expects JSON files to contain one JSON record per line.

When a file contains a single JSON document formatted across multiple lines, Spark may incorrectly parse the file unless multiline parsing is enabled.

Setting:

```
.option("multiline", "true")
```

allows Spark to correctly interpret multiline JSON documents during ingestion.

Question 38

A data engineering team has deployed a Lakeflow Spark Declarative Pipelines pipeline in continuous mode to process streaming IoT telemetry data.

The pipeline is designed to process new events with low latency as soon as they arrive. During a prolonged period of inactivity, no new events are generated by the source system.

Despite the lack of incoming data, the engineering team notices that compute costs continue to accumulate.

What is the most likely explanation for this behavior?

- A. Continuous pipeline mode keeps compute resources active even when no new data arrives
- B. The pipeline repeatedly reloads historical files from cloud storage during idle periods
- C. The pipeline is waiting for the next checkpoint commit before compute resources can shut down
- D. Auto Loader maintains excessive metadata state in executor memory during inactivity

Correct answer: A

- A. Continuous pipeline mode keeps compute resources active even when no new data arrives

Explanation

Continuous pipeline mode is designed for low-latency streaming workloads where newly arriving data should be processed as quickly as possible.

To maintain this low-latency behavior:

- compute resources remain active continuously
- the pipeline stays ready to process new data immediately
- costs continue even during idle periods with no incoming data

This execution model is appropriate for near real time streaming use cases, but it typically results in higher compute costs compared to triggered pipeline mode.

Triggered mode is often preferred for workloads where data arrives periodically and low-latency processing is not required.

Question 39

A data engineering team is building a Gold customer analytics table from a Silver transactions table named `retail.silver.transactions`.

The Silver table contains:

- `customer_id`
- `transaction_amount`

The business wants a Gold table that contains:

- the total number of transactions for each customer
- the average transaction amount for each customer

Which PySpark code snippet best meets these requirements?

A.

```
from pyspark.sql.functions import count, avg

gold_df = (
    spark.read.table("retail.silver.transactions")
    .groupBy("customer_id")
    .agg(
        count("*").alias("total_transactions"),
        avg("transaction_amount").alias("avg_transaction_amount")
    )
)
```

B.

```
from pyspark.sql.functions import count, avg

gold_df = (
    spark.read.table("retail.silver.transactions")
    .groupBy("customer_id")
    .agg(
        countDistinct("transaction_amount").alias("total_transactions"),
        avg("customer_id").alias("avg_transaction_amount")
    )
)
```

C.

```
from pyspark.sql.functions import count, avg

gold_df = (
    spark.read.table("retail.silver.transactions")
    .groupBy("transaction_amount")
    .agg(
        count("*").alias("total_transactions"),
        avg("transaction_amount").alias("avg_transaction_amount")
    )
)
```

```
)  
)
```

D.

```
from pyspark.sql.functions import count, avg  
  
gold_df = (  
    spark.read.table("retail.silver.transactions")  
    .agg(  
        count("*").alias("total_transactions"),  
        avg("transaction_amount").alias("avg_transaction_amount")  
    )  
)
```

Correct answer: A

Explanation

Aggregations are commonly used when transforming Silver data into Gold datasets for reporting and analytics.

In this scenario, the business requires customer-level metrics:

- total number of transactions per customer
- average transaction amount per customer

To generate these metrics, the data must first be grouped by `customer_id`. Aggregation functions such as `count()` and `avg()` can then be applied to calculate the required measures for each customer.

This pattern is commonly used when creating customer analytics, reporting, and business intelligence datasets in the Gold layer.

Question 40

A data engineering team is onboarding data from an external cloud storage into Unity Catalog.

The security team wants to:

- define the cloud authentication mechanism in Unity Catalog
- reuse the same authentication object across multiple containers in that storage account

Which Unity Catalog object best meets these requirements?

- A. External Storage
- B. Storage Credential
- C. Unity Catalog Volume
- D. External Location

Correct answer: B

B. Storage Credential

Explanation

Unity Catalog separates cloud authentication from storage path governance.

A **Storage Credential** defines how Unity Catalog authenticates to cloud storage, such as through a managed identity, IAM role, or service principal.

An **External Location** then combines:

- a Storage Credential
- a specific cloud storage path

This separation allows organizations to:

- centrally manage authentication
- reuse authentication across multiple storage containers and locations
- simplify credential rotation and security administration

In this scenario, the requirement is to define the authentication mechanism once and reuse it across multiple containers. A **Storage Credential** is specifically designed for this purpose.

Question 41

A healthcare organization stores patient records in a Unity Catalog table named `healthcare.silver.patient_records`.

The table contains data for multiple hospitals in a column named `hospital_id`.

The governance team wants to implement the following access policy:

- users in the `hospital_a_staff` group should see only records where `hospital_id = 'A'`
- users in the `hospital_b_staff` group should see only records where `hospital_id = 'B'`
- users in the `hospital_admins` group should see all records

Which SQL function best implements this row filter?

A.

```
CREATE FUNCTION hospital_row_filter(hospital_id STRING)
RETURN CASE
  WHEN is_account_group_member('hospital_admins') THEN hospital_id
  WHEN is_account_group_member('hospital_a_staff') THEN 'A'
  WHEN is_account_group_member('hospital_b_staff') THEN 'B'
  ELSE FALSE
END;
```

B.

```
CREATE FUNCTION hospital_row_filter(hospital_id STRING)
RETURN CASE
  WHEN is_account_group_member('hospital_admins') THEN TRUE
  WHEN is_account_group_member('hospital_a_staff') THEN hospital_id = 'A'
  WHEN is_account_group_member('hospital_b_staff') THEN hospital_id = 'B'
  ELSE FALSE
END;
```

C.

```
CREATE FUNCTION hospital_row_filter(hospital_id STRING)
RETURN CASE
  WHEN current_user() = 'hospital_admins' THEN TRUE
  WHEN current_user() = 'hospital_a_staff' THEN hospital_id = 'A'
  WHEN current_user() = 'hospital_b_staff' THEN hospital_id = 'B'
  ELSE FALSE
END;
```

D.

```
CREATE FUNCTION hospital_row_filter(hospital_id STRING)
RETURN CASE
  WHEN current_user('hospital_admins') THEN hospital_id
  WHEN current_user('hospital_a_staff') THEN 'A'
  WHEN current_user('hospital_b_staff') THEN 'B'
  ELSE FALSE
END;
```

Correct answer: B

Explanation

Row filters in Unity Catalog are used to restrict which rows users can see when querying a table.

The filter function is evaluated for every row:

- TRUE → the row is returned
- FALSE → the row is hidden

In this scenario, the organization wants different user groups to see different subsets of the same table while allowing administrators to view all records. A row filter function provides centralized row-level security without requiring separate tables or views for each hospital.

Question 42

A data engineering team is creating a Silver customer table from a Bronze table named `retail.bronze.customers`.

The Bronze table contains rows where `customer_id` is NULL.

The team wants to remove these records before writing the cleaned data to `retail.silver.customers`.

Which PySpark code snippet best meets these requirements?

A.

```
df = (
    spark.read.table("retail.bronze.customers")
    .filter(col("customer_id").isNotNull())
)

df.write.mode("overwrite").saveAsTable("retail.silver.customers")
```

B.

```
df = (
    spark.read.table("retail.bronze.customers")
    .filter(col("customer_id").isNull())
)

df.write.mode("overwrite").saveAsTable("retail.silver.customers")
```

C.

```
df = spark.read.table("retail.bronze.customers")

df.write.mode("overwrite").saveAsTable("retail.silver.customers")
```

D.

```
df = (
    spark.read.table("retail.bronze.customers")
    .drop("customer_id")
)

df.write.mode("overwrite").saveAsTable("retail.silver.customers")
```

Correct answer: A

Explanation

A common Bronze-to-Silver transformation pattern is to remove records that do not meet minimum data quality requirements before making the data available to downstream consumers.

In this scenario:

- `customer_id` is a required business key
- records with a missing `customer_id` should not be promoted to the Silver layer
- the goal is to filter out invalid records while preserving valid customer records

Using `isNotNull()` ensures that only records containing a valid customer identifier are written to the Silver table.

This type of cleansing operation is commonly performed during Bronze-to-Silver transformations to improve data quality before the data is used for reporting, analytics, or downstream processing.

Question 43

A data engineering team is troubleshooting a Spark workload that joins a very large transactions DataFrame with a relatively small lookup DataFrame containing product category mappings.

The engineering team notices that Spark is performing shuffle-heavy joins instead of distributing the smaller lookup table as a whole across executors. They want Spark to automatically apply this optimization to slightly larger lookup tables than it currently does.

Which Spark configuration should the team modify?

- A. `spark.default.parallelism`
- B. `spark.sql.shuffle.partitions`
- C. `spark.sql.autoBroadcastJoinThreshold`
- D. `spark.executor.memory`

Correct answer: C

C. `spark.sql.autoBroadcastJoinThreshold`

Explanation

Spark can optimize joins by distributing a smaller DataFrame to all executors instead of shuffling both sides of the join across the cluster.

The configuration:

```
spark.sql.autoBroadcastJoinThreshold
```

controls the maximum size of a DataFrame that Spark will automatically distribute using this optimization.

If the lookup DataFrame size is below this threshold:

- Spark avoids large shuffle operations
- network overhead is reduced
- join performance can improve significantly

Increasing this threshold allows Spark to automatically apply the optimization to larger lookup tables when sufficient executor memory is available.

Question 44

A data engineering team manages several production Delta tables that are updated frequently by automated pipelines.

The team needs the ability to:

- recover from accidental updates or deletions
- review earlier versions of tables
- audit historical data changes

Which Delta Lake capability best supports these requirements?

A. VACUUM

B. Photon

C. Time Travel

D. Delta Caching

Correct answer: C

C. Time Travel

Explanation

The scenario focuses on operational recovery and auditing requirements for production Delta tables.

Delta Lake Time Travel allows teams to:

- access historical versions of tables
- recover from accidental updates or deletions
- audit how data changed over time

This capability is especially important in production environments where pipelines continuously update data and teams may need to restore or inspect previous table states.

VACUUM removes older data files that are no longer required by Delta Lake. It is used for storage cleanup and retention management rather than historical table recovery. VACUUM should also be understood carefully because removing older data files can reduce the amount of history available for

Time Travel operations.

Question 45

A data engineering team is implementing a CI/CD workflow for deploying Declarative Automation Bundles to the production environment.

The engineering team wants the workflow to identify bundle configuration issues and deploy the bundle to production if there are no issues

Which sequence of Databricks CLI commands best meets these requirements?

- A.** databricks bundle validate
databricks bundle deploy --target prod
- B.** databricks bundle deploy --target prod
databricks bundle validate
- C.** databricks bundle [deploy, validate] --target prod
- D.** databricks bundle [validate, deploy] --target prod

Correct answer: A

A. databricks bundle validate
databricks bundle deploy --target prod

Explanation

A common Declarative Automation Bundle CI/CD workflow is:

- Validate the bundle configuration
- Identify configuration issues early
- Deploy the bundle after validation succeeds

The Databricks CLI commonly uses:

```
databricks bundle validate
```

followed by:

```
databricks bundle deploy --target <target_name>
```

This approach improves:

- deployment reliability
- CI/CD automation quality

- operational safety
- environment consistency

Validating before deployment helps identify:

- invalid configuration values
- missing deployment targets
- incorrect resource definitions
- environment configuration issues

Question 46

A data governance team wants to centrally enforce the following policies across multiple Unity Catalog tables:

- finance users should see unmasked financial data
- non-finance users should see masked values
- regional users should only see rows belonging to their assigned region
- policies should scale across many tables without manually configuring rules on each table individually

Which Unity Catalog capability best meets these requirements?

- A. Table ACLs
- B. Dynamic views
- C. Attribute-Based Access Control (ABAC) policies
- D. Role-Based Access Control (RBAC) policies

Correct answer: C

C. Attribute-Based Access Control (ABAC) policies

Explanation

ABAC policies are especially useful when:

- governance rules apply across many datasets
- row filtering and column masking must scale consistently
- policies depend on user or data attributes
- security logic should be centrally managed instead of recreated table by table

In this scenario:

- the governance team wants centralized policy enforcement
- policies should scale across many tables

- filtering and masking behavior should depend on user context and governed attributes

ABAC policies help reduce operational complexity by allowing governance teams to define reusable security policies based on governed tags and attributes.

Question 47

A retail company maintains a customer dimension table using Lakeflow Spark Declarative Pipelines.

Customer records are periodically updated from a CDC source system. The analytics team only needs the latest version of each customer record and does not need to preserve historical changes.

Which SQL definition best supports this requirement?

A.

```
CREATE OR REFRESH STREAMING TABLE dim_customers;
```

```
CREATE FLOW customer_flow AS  
AUTO CDC INTO dim_customers  
FROM stream(customers_bronze_cdc)  
KEYS (customer_id)  
SEQUENCE BY operation_timestamp  
STORED AS SCD TYPE 2;
```

B.

```
CREATE OR REFRESH STREAMING TABLE dim_customers;
```

```
CREATE FLOW customer_flow AS  
AUTO CDC INTO dim_customers  
FROM stream(customers_bronze_cdc)  
KEYS (customer_id)  
SEQUENCE BY operation_timestamp  
STORED AS SCD TYPE 1;
```

C.

```
CREATE OR REFRESH MATERIALIZED VIEW dim_customers AS  
SELECT * FROM customers_bronze_cdc;
```

D.

```
CREATE OR REFRESH STREAMING TABLE dim_customers AS  
SELECT DISTINCT * FROM customers_bronze_cdc;
```

Correct answer: B

Explanation

SCD Type 1 processing is used when only the latest version of a record should be stored.

In Lakeflow Spark Declarative Pipelines:

```
STORED AS SCD TYPE 1
```

tells `AUTO CDC INTO` to apply incoming CDC updates by replacing existing records that share the same business key.

For example, if a customer membership tier changes from:

Silver -> Gold

the existing customer record is updated with the new value rather than preserving the previous version.

This approach is commonly used when:

- historical tracking is not required
- downstream systems only need the current state of the data
- storage efficiency is preferred over maintaining historical versions

Question 48

A data engineering team is building a reporting pipeline using **Lakeflow Spark Declarative Pipelines**.

The team needs to maintain an aggregated dataset based on billions of records stored in a Delta table.

Business users run the same dashboard queries throughout the day.

The engineering team wants to:

- reduce query execution times
- avoid recalculating the aggregation every time users query the data
- keep the results automatically refreshed as source data changes

Which Lakeflow Spark Declarative Pipelines object best meets these requirements?

A. Materialized View

B. Streaming Table

C. Temporary View

D. Standard View

Correct answer: A

A. Materialized View

Explanation

Lakeflow Spark Declarative Pipelines supports both Materialized Views and Streaming Tables, each designed for different use cases.

Materialized Views are particularly useful when:

- complex aggregations are queried repeatedly
- query performance is important
- source data changes over time
- users benefit from precomputed results

Unlike a standard view, which recomputes results whenever it is queried, a Materialized View stores and maintains the computed results. As source data changes, Lakeflow Spark Declarative Pipelines automatically refreshes the Materialized View.

In this scenario, business users repeatedly execute dashboard queries against a large aggregated dataset. A Materialized View helps reduce query latency while ensuring the results remain up to date.

Question 49

A data engineering team is troubleshooting a slow Lakeflow Job that processes a large fact table join.

While reviewing the Spark UI, the engineering team notices:

- high spill-to-disk metrics during shuffle stages
- increased disk I/O
- failed tasks during shuffle processing

Which issue is the most likely cause of this behavior?

- A.** Excessive task dependency in the Lakeflow Job.
- B.** Cluster node failures.
- C.** Large number of shuffle partitions.
- D.** Out-of-memory issues caused by insufficient executor memory.

Correct answer: D

D. Out-of-memory issues caused by insufficient executor memory.

Explanation

Out-of-memory issues are a common Spark performance bottleneck during shuffle-heavy operations such as:

- joins
- aggregations
- sorts

When executors do not have enough memory to process intermediate shuffle data in memory, Spark spills data to disk, increasing disk I/O and slowing execution.

Typical Spark UI symptoms include:

- spill-to-disk activity
- increased disk I/O

- failed shuffle tasks
- shuffle-related performance degradation

The Spark UI helps engineering teams diagnose memory pressure and shuffle-related bottlenecks through stage-level metrics and task execution details.

Question 50

A retail company maintains two Silver tables:

- online_customers containing customers who have placed online orders
- store_customers containing customers who have made in-store purchases

The analytics team wants to create a Gold dataset that includes **all customers from both tables, regardless of whether a matching customer exists in the other table.**

Which PySpark join best meets this requirement?

A.

```
gold_df = online_customers.join(
    store_customers,
    "customer_id",
    "full"
)
```

B.

```
gold_df = online_customers.join(
    store_customers,
    "customer_id",
    "inner"
)
```

C.

```
gold_df = online_customers.join(
    store_customers,
    "customer_id",
    "right"
)
```

D.

```
gold_df = online_customers.join(
    store_customers,
    "customer_id",
    "left"
)
```

Correct answer: A

Explanation

A full outer join is used when all records from both datasets must be retained, regardless of whether matching keys exist in the other dataset. This type of join is commonly used when building consolidated Gold datasets that combine information from multiple business channels while preserving complete coverage of entities across all source systems.

Question 51

Two data engineers are working on the same notebook inside a Databricks Git Folder connected to a shared Git repository.

One engineer commits and pushes changes to the remote branch. The second engineer, unaware of the new commits, attempts to push their own notebook changes without pulling the latest updates first.

The second engineer receives an error indicating that their branch is behind the remote branch.

Why is this error occurring, and how should the engineer resolve it?

- A.** Databricks Git Folders do not support multiple engineers editing the same notebook simultaneously, so the notebook must be locked before changes can be pushed.
- B.** The notebook was committed using incorrect Git credentials, so the engineer must reconnect the Databricks Git Folder to the repository.
- C.** The engineer attempted to push notebook changes from a cluster that was not synchronized with the Databricks control plane.
- D.** The remote branch contains newer commits that are missing locally, so the engineer must pull the latest changes and resolve any merge conflicts before pushing again.

Correct answer: D

D. The remote branch contains newer commits that are missing locally, so the engineer must pull the latest changes and resolve any merge conflicts before pushing again.

Explanation

Git prevents engineers from pushing changes when their local branch is behind the remote branch.

This behavior helps prevent:

- accidental overwriting of other engineers' work
- inconsistent branch history
- loss of committed changes

A common collaborative workflow is:

- Pull the latest changes from the remote branch
- Resolve merge conflicts if necessary

- Commit any conflict resolutions
- Push the updated branch

Merge conflicts commonly occur when multiple engineers modify the same notebook, file, or code section concurrently.

Understanding pull, merge, and conflict resolution workflows is important for collaborative CI/CD development using Databricks Git Folders.

Question 52

A data engineering team is developing ETL pipelines in Databricks.

Several engineers work collaboratively in notebooks throughout the day.

The team requires a compute environment that supports:

- interactive notebook development
- multiple users attaching to the same compute
- shared library configurations
- collaborative development workflows

Which compute option is most appropriate?

- A.** Serverless compute
- B.** Job compute
- C.** SQL warehouse
- D.** All-purpose compute

Correct answer: D

D. All-purpose compute

Explanation

Different Databricks compute options are optimized for different workload types.

In this scenario, the team requires a shared environment for collaborative ETL development. Multiple engineers need to work interactively in notebooks while using the same runtime configuration and library setup.

All-purpose compute is designed for these development workflows because it:

- supports interactive notebook development
- allows multiple users to attach to the same compute
- provides a consistent shared runtime environment
- supports installation of custom libraries and configuration changes

While Serverless Compute reduces operational overhead through a fully managed experience, it uses the latest supported Databricks Runtime and a predefined set of libraries managed by Databricks. When teams require greater control over runtime configuration and library management during development, All-purpose Compute is generally the more appropriate choice.

Question 53

A central platform team maintains a certified `customer_profile` delta table that is refreshed by a separate upstream pipeline whenever CRM data changes.

A downstream marketing analytics team has its own Lakeflow Job that builds campaign segmentation tables using data from `customer_profile`.

The marketing team does not control the upstream pipeline schedule and wants their job to:

- execute as soon as `customer_profile` is updated
- ensure downstream processing uses the latest certified customer data

Which trigger type best meets these requirements?

- A.** Configure the Lakeflow Job to run on a continuous trigger to process the table as soon as it is refreshed.
- B.** Configure the Lakeflow Job using a scheduled hourly trigger.
- C.** Configure the Lakeflow Job using a table update trigger on the `customer_profile` table.
- D.** Configure the Lakeflow Job using a file arrival trigger on the storage location of the `customer_profile` table.

Correct answer: C

- C.** Configure the Lakeflow Job using a table update trigger on the `customer_profile` table.

Explanation

Lakeflow Jobs supports multiple trigger types for workflow orchestration, including:

- scheduled triggers
- file arrival triggers
- table update triggers

Table update triggers are designed for data-driven orchestration scenarios where downstream jobs should execute automatically after upstream tables are refreshed.

In this scenario:

- the upstream `customer_profile` table is maintained independently by another team
- downstream processing should start automatically after table updates occur
- downstream analytics should always use the latest certified customer data

Using a table update trigger improves:

- downstream data freshness
- workflow automation
- dependency-driven orchestration
- coordination between independently managed pipelines

This pattern is commonly used when multiple independent teams or pipelines share certified upstream datasets.

Question 54

A company currently uses a Hive metastore to manage tables across multiple Databricks workspaces. Different teams manage permissions independently, making it difficult to enforce consistent governance policies and track data lineage centrally.

The company wants to:

- centralize access control
- simplify governance management across workspaces
- improve visibility into data lineage and auditing

Which solution best addresses these requirements?

- A.** Use Delta Live Tables for all ingestion pipelines
- B.** Migrate to Unity Catalog
- C.** Enable Photon on existing compute resources
- D.** Use Delta Lake tables for all workloads

Correct answer: B

B. Migrate to Unity Catalog

Explanation

The company's main challenge is fragmented governance across multiple Databricks workspaces using a Hive metastore-based approach.

Unity Catalog provides a centralized governance layer that allows organizations to:

- manage permissions consistently across workspaces
- track lineage centrally
- improve auditing visibility
- simplify governance administration

This makes Unity Catalog the best solution for organizations that want centralized governance and standardized access management across the Databricks platform.

Question 55

A data engineering team is building a Silver orders table using Lakeflow Spark Declarative Pipelines.

The Bronze orders table contains some records with:

- NULL order_id values
- negative quantity values

The team wants to:

- continue processing all records into the Silver table
- track both data quality issues independently in pipeline metrics
- avoid failing the pipeline update

Which TWO code snippets best meet these requirements?

A.

```
@dp.table
@dp.expect("valid_order_id", "order_id IS NOT NULL")
@dp.expect("valid_quantity", "quantity > 0")
def silver_orders():
    return spark.read.table("formula1.bronze.orders")
```

B.

```
@dp.table
@dp.expect_or_warn("valid_order_id", "order_id IS NOT NULL")
@dp.expect_or_warn("valid_quantity", "quantity > 0")
def silver_orders():
    return spark.read.table("formula1.bronze.orders")
```

C.

```
rules = {
    "valid_order_id": "order_id IS NOT NULL",
    "valid_quantity": "quantity > 0"
}
```

```
@dp.table
@dp.expect_all(rules)
def silver_orders():
    return spark.read.table("formula1.bronze.orders")
```

D.

```
@dp.table
@dp.expect("valid_order_id", "order_id IS NULL")
@dp.expect("valid_quantity", "quantity < 0")
def silver_orders():
    return spark.read.table("formula1.bronze.orders")
```

Correct answers: A, C

Explanation

Lakeflow Spark Declarative Pipelines supports multiple expectation patterns for tracking data quality issues during pipeline execution.

When records should continue flowing through the pipeline while still generating quality metrics, teams can use:

- multiple `@dp.expect()` decorators
- `@dp.expect_all()` with grouped rules

Both approaches allow data quality monitoring without dropping records or failing the pipeline update.